

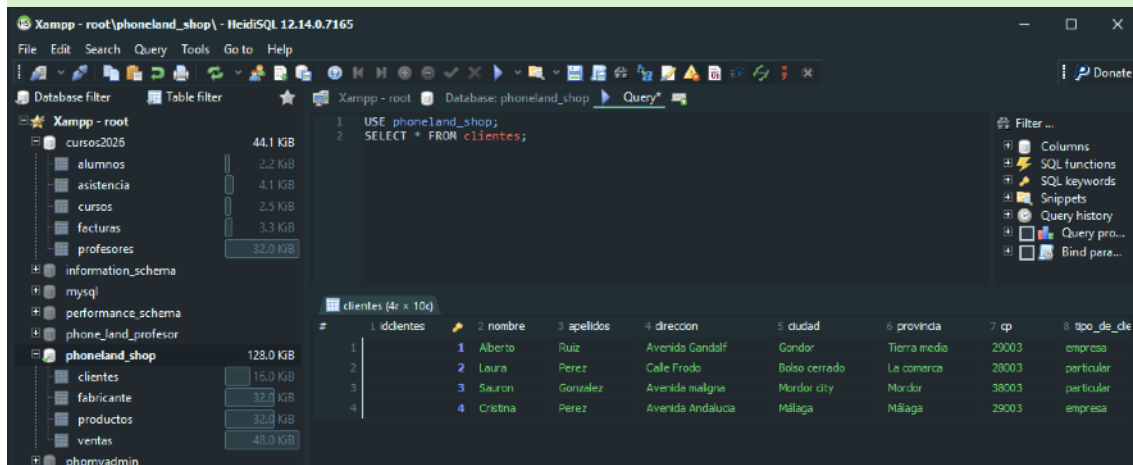
EJERCICIOS BASE DE DATOS PHONELAND

SOLUCIONES — BLOQUE A (SQL básico)

A1. SELECT y columnas

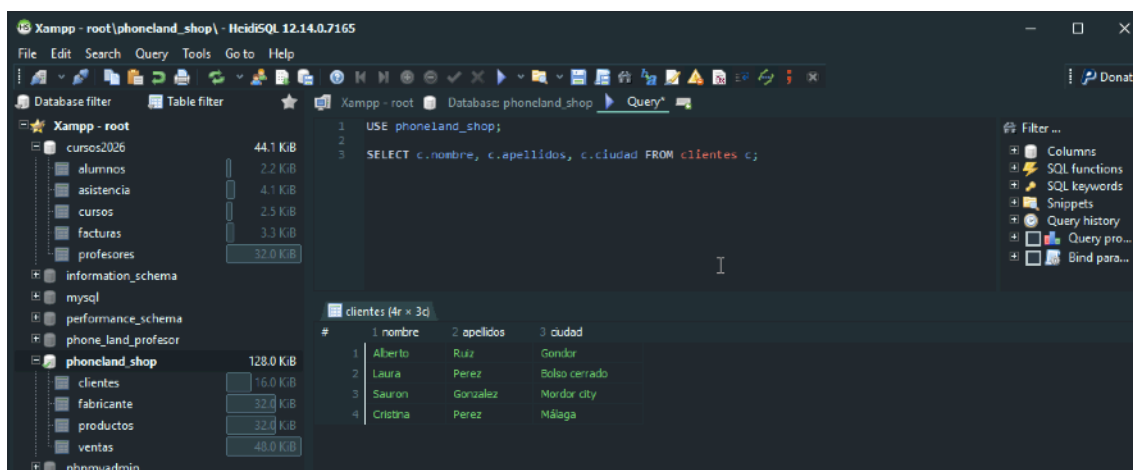
1) Todas las columnas de clientes

```
SELECT * FROM clientes;
```



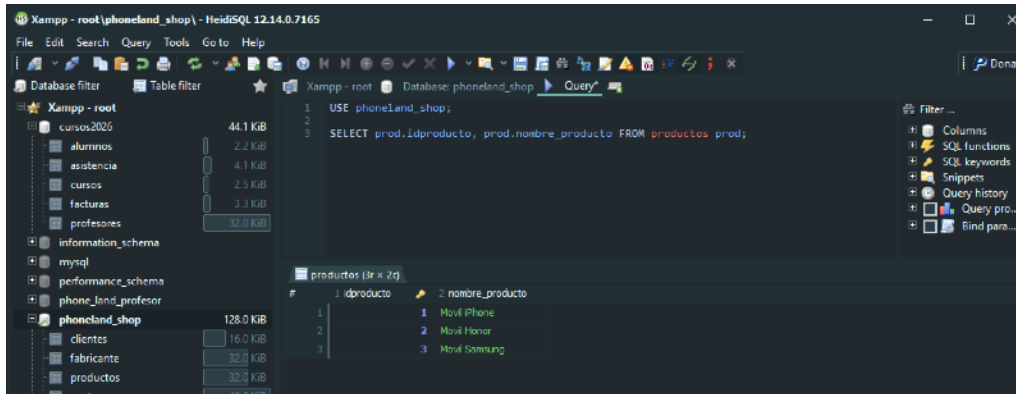
2) nombre, telefono, email

```
SELECT nombre, telefono, email  
FROM clientes;
```



3) id y nombre de productos

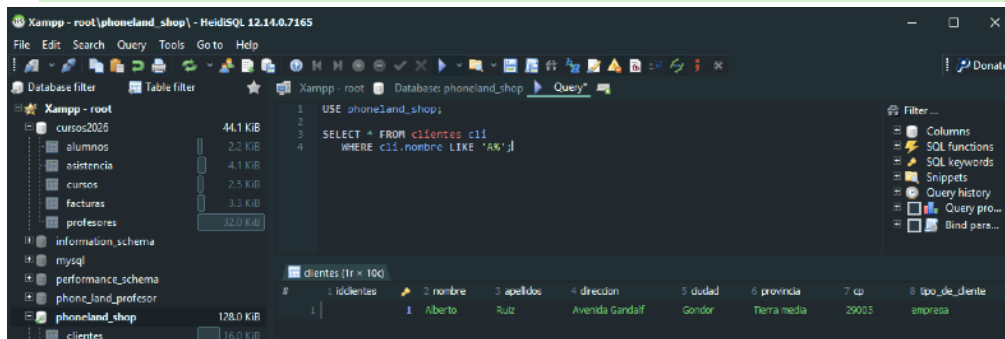
```
SELECT id_producto, nombre  
FROM productos;
```



A2. WHERE (filtros)

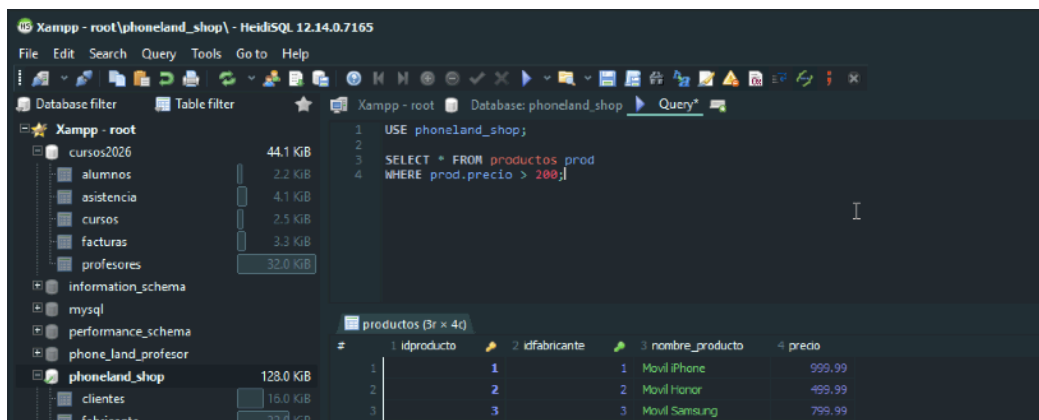
4) Clientes cuyo nombre empieza por A

```
SELECT *  
FROM clientes  
WHERE nombre LIKE 'A%';
```



5) Productos con precio > 200

```
SELECT *  
FROM productos  
WHERE precio > 200;
```



6) Ventas de una fecha concreta

```
SELECT *  
FROM ventas  
WHERE fecha = '2025-01-15';
```

The screenshot shows the HeidiSQL interface with the following details:

- Top Bar:** Xampp - root | phoneland_shop | ventas | - HeidiSQL 12.14.0.7165
- Menu Bar:** File Edit Search Query Tools Goto Help
- Database Filter:** Xampp - root
 - curso2026 (44.1 KB)
 - alumnos (2.2 KB)
 - asistencia (4.1 KB)
 - curso (2.5 KB)
 - facturas (3.3 KB)
 - profesores (32.0 KB)
 - information_schema
 - mysql
 - performance_schema
 - phone_lend_profesor
 - phoneland_shop (128.0 KB)
 - clientes (16.0 KB)
- Query Editor:**

```

1 USE phoneland_shop;
2
3 SELECT * FROM ventas v
4 WHERE v.fecha_ventas = '2023-01-10';
      
```
- Query Results:**

ventas (tr = 4)

#	1 idventas	2 ididentes	3 idproductos	4 fecha_ventas
1	1	1	1	2023-01-10
- Right Panel:** Filter ...
 - Columns in v...
 - SQL functions
 - SQL keywords
 - Snippets
 - Query history
 - Query pro...
 - Bind para...

7) Clientes con email NULL

```
SELECT *  
FROM clientes  
WHERE email IS NULL;
```

The screenshot shows the MySQL Workbench interface. On the left, the 'Database filter' pane displays the schema structure for 'Xampp - root'. The 'phoneland_shop' database is expanded, showing tables: 'clientes' (16.0 KiB), 'profesores' (32.0 KiB), 'facturas' (3.3 KiB), 'cursos' (2.5 KiB), 'asistencia' (4.1 KiB), 'alumnos' (2.2 KiB), and 'cursos2025' (44.1 KiB). The 'clientes' table is selected.

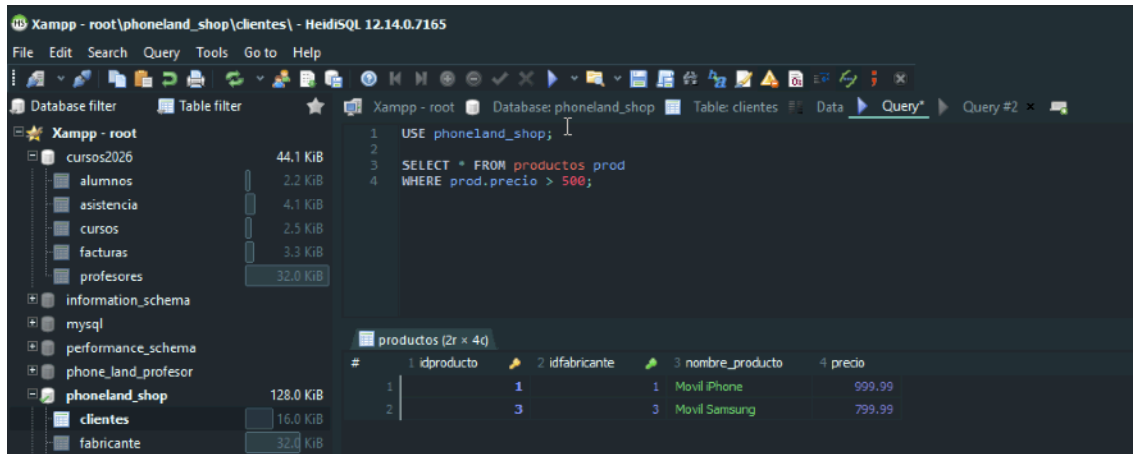
The central pane shows the SQL editor with the following query:

```
1  USE phoneland_shop;
2
3  SELECT * FROM clientes cli
4  WHERE cli.tipo_de_cliente IS NULL;
```

The bottom pane shows the 'clientes (0r x 10c)' table structure with the following columns: 1 id_clientes, 2 nombre, 3 apellidos, 4 direccion, 5 ciudad, 6 provincia, 7 cp, 8 tipo_de_cliente, and 9 ingresos.

8) Productos con stock = 0

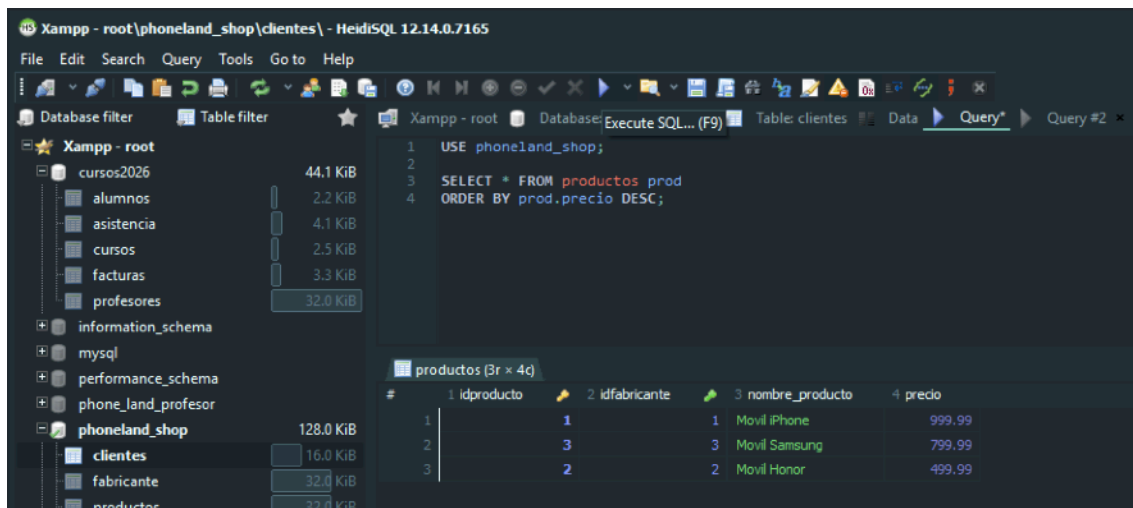
```
SELECT *  
FROM productos  
WHERE stock = 0;
```



A3. ORDER BY y LIMIT

9) Productos ordenados por precio desc

```
SELECT *  
FROM productos  
ORDER BY precio DESC;
```



10) 5 productos más caros

```
SELECT *  
FROM productos  
ORDER BY precio DESC  
LIMIT 5;
```

The screenshot shows the HeidiSQL interface with the query executed. The results table, titled 'productos (3r x 4c)', displays the following data:

#	1 idproducto	2 idfabricante	3 nombre_producto	4 precio
1	1	1	Movil iPhone	999.99
2	2	3	Movil Samsung	799.99
3	3	2	Movil Honor	499.99

11) Clientes ordenados alfabéticamente

```
SELECT *  
FROM clientes  
ORDER BY nombre ASC;
```

The screenshot shows the HeidiSQL interface with the query executed. The results table, titled 'clientes (4r x 10c)', displays the following data:

#	1 idclientes	2 nombre	3 apellidos	4 direccion	5 ciudad	6 provincia	7 cp	8 tipo_de_cliente
1	1	Alberto	Ruiz	Avenida Gandalf	Gondor	Tierra media	29003	empresa
2	4	Cristina	Perez	Avenida Andalucía	Málaga	29003	empresa	empresa
3	2	Laura	Perez	Calle Frodo	Bolso cerrado	La comarca	28003	particular
4	3	Sauron	Gonzalez	Avenida maligna	Mordor city	Mordor	38003	particular

A4. Agregaciones

12) Número de clientes

```
SELECT COUNT(*) AS total_clientes  
FROM clientes;
```

The screenshot shows the HeidiSQL interface with the following details:

- Database filter:** Xampp - root
- Table filter:** Table: clientes
- Query Editor:**

```
1 USE phoneland_shop;  
2  
3 SELECT COUNT(cli.idclientes) AS total_cli  
4 FROM clientes cli  
5
```
- Result Set:**

#	1 total_cli
1	4
- Database Structure:**
 - courses2026 (44.1 KiB)
 - alumnos (2.2 KiB)
 - asistencia (4.1 KiB)
 - courses (2.5 KiB)
 - facturas (3.3 KiB)
 - profesores (32.0 KiB)
 - information_schema
 - mysql
 - performance_schema
 - phone_land_profesor
 - phoneland_shop (128.0 KiB)
 - clientes (16.0 KiB)
 - fabricante (32.0 KiB)

13) Número de productos

```
SELECT COUNT(*) AS total_productos  
FROM productos;
```

The screenshot shows the HeidiSQL interface with the following details:

- Database filter:** Xampp - root
- Table filter:** Table: productos
- Query Editor:**

```
1 USE phoneland_shop;  
2  
3 SELECT COUNT(prod.idproductd) AS total_prod  
4 FROM productos prod  
5
```
- Result Set:**

#	1 total_prod
1	3
- Database Structure:**
 - courses2026 (44.1 KiB)
 - alumnos (2.2 KiB)
 - asistencia (4.1 KiB)
 - courses (2.5 KiB)
 - facturas (3.3 KiB)
 - profesores (32.0 KiB)
 - information_schema
 - mysql
 - performance_schema
 - phone_land_profesor
 - phoneland_shop (128.0 KiB)
 - clientes (16.0 KiB)
 - fabricante (32.0 KiB)
 - productos (32.0 KiB)

14) Precio medio

```
SELECT AVG(precio) AS precio_medio  
FROM productos;
```

15) Total de ventas (si existe campo total)

```
SELECT SUM(total) AS total_facturado  
FROM ventas;
```

Si NO existe total, luego lo calculamos con detalle_venta (te lo doy abajo).

A5. GROUP BY + HAVING

16) Número de ventas por cliente

```
SELECT id_cliente, COUNT(*) AS num_ventas  
FROM ventas  
GROUP BY id_cliente;
```

17) Número de ventas por fecha

```
SELECT fecha, COUNT(*) AS num_ventas  
FROM ventas  
GROUP BY fecha  
ORDER BY fecha;
```

18) Clientes con más de 1 compra

```
SELECT id_cliente, COUNT(*) AS num_ventas  
FROM ventas  
GROUP BY id_cliente  
HAVING COUNT(*) > 1;
```

✅ SOLUCIONES — BLOQUE B (INNER JOIN)

B1. JOIN básico

1) Ventas con nombre del cliente

```
SELECT v.id_venta, v.fecha, c.nombre AS cliente  
FROM ventas v  
INNER JOIN clientes c ON v.id_cliente = c.id_cliente;
```

2) Ventas con id_venta, fecha, cliente ordenadas por fecha

```
SELECT v.id_venta, v.fecha, c.nombre AS cliente
FROM ventas v
INNER JOIN clientes c ON v.id_cliente = c.id_cliente
ORDER BY v.fecha;
```

3) Ventas de un cliente concreto (por id)

```
SELECT v.*
FROM ventas v
WHERE v.id_cliente = 3;
(Alternativa por nombre)
SELECT v.*
FROM ventas v
INNER JOIN clientes c ON v.id_cliente = c.id_cliente
WHERE c.nombre = 'Ana';
```

B2. JOIN con 3 tablas

4) Detalle: venta + cliente + producto + cantidad

```
SELECT v.id_venta, v.fecha, c.nombre AS cliente, p.nombre AS
producto, dv.cantidad
FROM ventas v
INNER JOIN clientes c ON v.id_cliente = c.id_cliente
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta
INNER JOIN productos p ON dv.id_producto = p.id_producto;
```

5) Detalle de una venta concreta

```
SELECT v.id_venta, v.fecha, c.nombre AS cliente, p.nombre AS
producto, dv.cantidad, dv.precio_unitario
FROM ventas v
INNER JOIN clientes c ON v.id_cliente = c.id_cliente
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta
INNER JOIN productos p ON dv.id_producto = p.id_producto
```

```
WHERE v.id_venta = 10;
```

B3. JOIN con cálculo

6) Importe por línea

```
SELECT dv.id_venta, p.nombre AS producto, dv.cantidad,  
       dv.precio_unitario,  
       (dv.cantidad * dv.precio_unitario) AS importe_linea  
FROM detalle_venta dv  
INNER JOIN productos p ON dv.id_producto = p.id_producto;
```

7) Total de cada venta sumando líneas

```
SELECT dv.id_venta,  
       SUM(dv.cantidad * dv.precio_unitario) AS total_venta  
FROM detalle_venta dv  
GROUP BY dv.id_venta;
```

8) Ventas con total > 300

```
SELECT dv.id_venta,  
       SUM(dv.cantidad * dv.precio_unitario) AS total_venta  
FROM detalle_venta dv  
GROUP BY dv.id_venta  
HAVING total_venta > 300;
```

B4. JOIN + GROUP BY

9) Gasto total por cliente

```
SELECT c.id_cliente, c.nombre,  
       SUM(dv.cantidad * dv.precio_unitario) AS gasto_total  
FROM clientes c  
INNER JOIN ventas v ON c.id_cliente = v.id_cliente  
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta  
GROUP BY c.id_cliente, c.nombre;
```

10) Top 5 clientes que más han gastado

```
SELECT c.id_cliente, c.nombre,
```

```
SUM(dv.cantidad * dv.precio_unitario) AS gasto_total
FROM clientes c
INNER JOIN ventas v ON c.id_cliente = v.id_cliente
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta
GROUP BY c.id_cliente, c.nombre
ORDER BY gasto_total DESC
LIMIT 5;
```

11) Producto más vendido (por cantidad)

```
SELECT p.id_producto, p.nombre,
       SUM(dv.cantidad) AS unidades_vendidas
FROM productos p
INNER JOIN detalle_venta dv ON p.id_producto = dv.id_producto
GROUP BY p.id_producto, p.nombre
ORDER BY unidades_vendidas DESC
LIMIT 1;
```

12) Ranking productos vendidos

```
SELECT p.id_producto, p.nombre,
       SUM(dv.cantidad) AS unidades_vendidas
FROM productos p
INNER JOIN detalle_venta dv ON p.id_producto = dv.id_producto
GROUP BY p.id_producto, p.nombre
ORDER BY unidades_vendidas DESC;
```

SOLUCIONES — BLOQUE C (Subconsultas)

C1. IN

1) Clientes que han comprado alguna vez

```
SELECT *
FROM clientes
WHERE id_cliente IN (SELECT id_cliente FROM ventas);
```

2) Productos que aparecen en alguna venta

```
SELECT *  
FROM productos  
WHERE id_producto IN (SELECT id_producto FROM detalle_venta);
```

C2. NOT IN

3) Clientes que nunca han comprado

```
SELECT *  
FROM clientes  
WHERE id_cliente NOT IN (SELECT id_cliente FROM ventas);
```

4) Productos nunca vendidos

```
SELECT *  
FROM productos  
WHERE id_producto NOT IN (SELECT id_producto FROM  
detalle_venta);
```

C3. Subconsulta escalar

5) Productos con precio superior a la media

```
SELECT *  
FROM productos  
WHERE precio > (SELECT AVG(precio) FROM productos);
```

6) Ventas con total superior a la media (calculando total por venta)

```
SELECT t.id_venta, t.total_venta  
FROM (  
    SELECT dv.id_venta, SUM(dv.cantidad * dv.precio_unitario) AS total_venta  
    FROM detalle_venta dv  
    GROUP BY dv.id_venta  
) t  
WHERE t.total_venta > (  
    SELECT AVG(x.total_venta)  
    FROM (
```

```

SELECT dv.id_venta, SUM(dv.cantidad * dv.precio_unitario) AS total_venta
FROM detalle_venta dv
GROUP BY dv.id_venta
) x
);

```

C4. EXISTS / NOT EXISTS

7) Clientes para los que existe al menos una venta

```

SELECT c.*
FROM clientes c
WHERE EXISTS (
    SELECT 1
    FROM ventas v
    WHERE v.id_cliente = c.id_cliente
);

```

8) Productos para los que NO existe detalle de venta

```

SELECT p.*
FROM productos p
WHERE NOT EXISTS (
    SELECT 1
    FROM detalle_venta dv
    WHERE dv.id_producto = p.id_producto
);

```

C5. Correlacionadas (nivel pro)

9) Última fecha de compra de cada cliente

```

SELECT c.id_cliente, c.nombre,
    (SELECT MAX(v.fecha)
     FROM ventas v
     WHERE v.id_cliente = c.id_cliente) AS ultima_compra
FROM clientes c;

```

10) Producto más caro de cada categoría (si existe categoria_id)

```
SELECT p.*  
FROM productos p  
WHERE p.precio = (  
    SELECT MAX(p2.precio)  
    FROM productos p2  
    WHERE p2.id_categoria = p.id_categoria  
);
```

BONUS “Modo examen” — Soluciones

1) Clientes con gasto > 500

```
SELECT c.id_cliente, c.nombre,  
       SUM(dv.cantidad * dv.precio_unitario) AS gasto_total  
FROM clientes c  
INNER JOIN ventas v ON c.id_cliente = v.id_cliente  
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta  
GROUP BY c.id_cliente, c.nombre  
HAVING gasto_total > 500;
```

2) Productos vendidos en 2025 y unidades

```
SELECT p.nombre, SUM(dv.cantidad) AS unidades  
FROM ventas v  
INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta  
INNER JOIN productos p ON dv.id_producto = p.id_producto  
WHERE v.fecha >= '2025-01-01' AND v.fecha < '2026-01-01'  
GROUP BY p.nombre  
ORDER BY unidades DESC;
```

3) Clientes que compraron un producto concreto

```
SELECT DISTINCT c.id_cliente, c.nombre  
FROM clientes c
```

INNER JOIN ventas v ON c.id_cliente = v.id_cliente

INNER JOIN detalle_venta dv ON v.id_venta = dv.id_venta

INNER JOIN productos p ON dv.id_producto = p.id_producto

WHERE p.nombre = 'iPhone';